

Helm

Packaging and delivery for Kubernetes

Author: Aliaksandr Tsimokhau



Agenda

Part 1: Helm Basics

Chart structure, subcharts, dependencies, release model

Part 2: Template Syntax

Operators, functions, flow control, variables

Part 3: Best Practices

Partials, naming, docs, values & packaging

Part 4: Advanced Patterns

Hooks, CD pipelines, versioning, OCI registries

Part 5: Helm 4 — what's new

WASM plugins, SSA, OCI digests, multi-doc values

Part 6: Lab

Практика — создаём первый Helm chart

Part 1: Helm Basics

Chart structure, subcharts, and dependencies

Why Helm? (1/2)

- **Chart** groups related Kubernetes manifests into one deployable package
- **Templates + values** keep configuration out of raw YAML duplication
- **Dependencies** let you compose one app from reusable subcharts
- **Release** captures the installed state of a chart and its config

Why Helm? (2/2)

- **Application-level view** of a Kubernetes cluster
- **Lifecycle management** for install, upgrade, rollback and uninstall
- **Built-in rollback** to a previous revision without rebuilding artifacts
- **OCI-based registries** are the recommended way to store and share chart packages [\[1\]](#)

Chart structure

Helm creates the chart scaffold in one command:

```
helm create nginx
```

```
nginx/
├── Chart.yaml           # chart metadata
├── README.md           # OPTIONAL: human-readable docs
├── LICENSE              # OPTIONAL: license file
├── crds/               # Custom Resource Definitions
├── charts/             # sub-charts (dependencies)
├── templates/
│   ├── NOTES.txt       # OPTIONAL: post-install notes
│   ├── _helpers.tpl    # reusable template helpers
│   ├── deployment.yaml
│   ├── ingress.yaml
│   └── service.yaml
└── values.yaml         # default configuration values
```

Chart anatomy

Chart.yaml

Name, version, dependencies, metadata

values.yaml

Default config for every install

templates/

Rendered manifests and helpers

crds/ + charts/

CRDs and dependencies shipped with the chart

`helm create` gives you the skeleton, but you still own the design

What is reserved

`charts/` - dependencies and subcharts

`crds/` - installed before templates render

`templates/` - rendered into Kubernetes manifests

`values.schema.json` - validation for values

Subcharts — umbrella chart structure

```
backend-chart/  
  Chart.yaml  
  charts/  
    message-queue/  
      Chart.yaml  
      templates/  
      values.yaml  
    database/  
      Chart.yaml  
      templates/  
      values.yaml  
  values.yaml
```


Subcharts — Chart.yaml dependencies

```
apiVersion: v2
name: backend-chart
description: A Helm chart for backend

dependencies:
- name: redis
  version: 20.8.0
  repository: oci://registry-1.docker.io/bitnamicharts
  alias: cache
  condition: cache.enabled
  tags:
  - backend
```

Subcharts — overriding values

Override subchart values in the parent `values.yaml`:

```
message-queue:
  enabled: true
  image:
    repository: acme/rabbitmq
    tag: latest
database:
  enabled: false
```

Useful debugging commands

Command	Purpose
helm lint --strict	Validate chart structure, syntax, and best-practice violations
helm template --debug	Render templates locally without touching the cluster
helm install --dry-run --debug	Simulate install and show the rendered release
helm show values <chart>	Inspect the default values shipped by the chart
helm dependency update	Download or refresh chart dependencies
helm get manifest <release>	Show manifests of installed resources
helm test <release>	Run chart test hooks after install or upgrade

Part 2: Template Syntax

Operators, functions, flow control, variables

Operators — boolean

Operator	Description	Example
and	Returns first empty arg or last arg	and .Arg1 .Arg2
or	Returns first non-empty arg or last arg	or .Arg1 .Arg2
not	Boolean negation	not .Arg

In Helm templates, operators (`eq` , `ne` , `lt` , `gt` , `and` , `or`) are implemented as functions. Operations can be grouped with parentheses (`( )`).

Operators — comparisons

Operator	Description	Example
eq	Arg1 == Arg2	eq .Arg1 .Arg2
ne	Arg1 != Arg2	ne .Arg1 .Arg2
lt	Arg1 < Arg2	lt .Arg1 .Arg2
le	Arg1 <= Arg2	le .Arg1 .Arg2
gt	Arg1 > Arg2	gt .Arg1 .Arg2
ge	Arg1 >= Arg2	ge .Arg1 .Arg2

String functions

Helm includes the following string functions:

`abbrev`, `abbrevboth`, `camelcase`, `cat`, `contains`, `hasPrefix`, `hasSuffix`, `indent`, `initials`, `kebabcase`,
`lower`, `nindent`, `nospace`, `plural`, `print`, `printf`, `println`, `quote`, `randAlpha`, `randAlphaNum`, `randAscii`,
`randNumeric`, `repeat`, `replace`, `shuffle`, `snakecase`, `squote`, `substr`, `swapcase`, `title`, `trim`, `trimAll`,
`trimPrefix`, `trimSuffix`, `trunc`, `untitle`, `upper`, `wrap`, `wrapWith`

Template syntax — default

To set a fallback value, use `default` :

```
default "foo" .Bar
```

```
{{ default "minio" .Values.storage }}  
  
// same as pipeline form:  
{{ .Values.storage | default "minio" }}
```


Flow control

- `if/else` — conditional blocks
- `with` — restrict scope to a sub-object
- `range` — `for each` -style loop

Flow control — `if/else`

```
{{ if PIPELINE }}  
  # Do something  
{{ else if OTHER PIPELINE }}  
  # Do something else  
{{ else }}  
  # Default case  
{{ end }}
```

```
data:  
  myvalue: "Hello World"  
  drink: {{ .Values.favorite.drink  
           | default "tea" | quote }}  
  food: {{ .Values.favorite.food  
           | upper | quote }}  
  {{ if eq .Values.favorite.drink "coffee" }}  
    mug: "true"  
  {{ end }}
```

Flow control — `if/else` with TLS

```
ports:
- port: {{ .Values.service.externalPort }}
  targetPort: {{ .Values.service.internalPort }}
  protocol: TCP
{{- if .Values.service.tls }}
  name: "https-{{ .Values.service.name }}"
{{- end }}
```

Service scheme and ports:

```
type: {{ .Values.service.type }}
ports:
- port: {{ .Values.service.externalPort }}
  targetPort: {{ .Values.service.internalPort }}
  protocol: TCP
{{- if .Values.service.tls }}
  name: "https-{{ .Values.service.name }}"
{{- else }}
  name: "{{ .Values.service.name }}"
{{- end }}
```

Flow control — `with`

`with` re-scopes `.` to a sub-object:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ .Release.Name }}-configmap
data:
  myvalue: "Hello World"
  {{- with .Values.favorite }}
  drink: {{ .drink | default "tea" | quote }}
  food: {{ .food | upper | quote }}
  {{- end }}
```

Notice that `.drink` and `.food` are referenced without qualifying them — `with` sets `.` to point to `.Values.favorite`. The `.` is reset to its previous scope after `{{ end }}`.

Flow control — `with` scope pitfall

This **fails** — `Release.Name` is outside the restricted scope:

```
{{- with .Values.favorite }}  
  drink: {{ .drink | default "tea" | quote }}  
  food: {{ .food | upper | quote }}  
  release: {{ .Release.Name }}  
{{- end }}
```

Workaround — move `release` outside `with`:

```
{{- with .Values.favorite }}  
  drink: {{ .drink | default "tea" | quote }}  
  food: {{ .food | upper | quote }}  
{{- end }}  
release: {{ .Release.Name }}
```

Flow control — `with` and `$`

Use `$` to access root scope from inside `with`:

```
{{- with .Values.favorite }}  
  drink: {{ .drink | default "tea" | quote }}  
  food: {{ .food | upper | quote }}  
  release: {{ $.Release.Name }}  
{{- end }}
```

`$` is mapped to the root scope when template execution begins and never changes during execution

Flow control — **range** setup

```
# values.yaml
favorite:
  drink: coffee
  food: pizza
pizzaToppings: |-
  - mushrooms
  - cheese
  - peppers
  - onions
```

The `|-` YAML marker creates a multi-line string — useful for embedding large data blocks

Flow control — range loop

```
data:
  myvalue: "Hello World"
  {{- with .Values.favorite }}
  drink: {{ .drink | default "tea" | quote }}
  food: {{ .food | upper | quote }}
  {{- end }}
  toppings: |-
    {{- range .Values.pizzaToppings }}
    - {{ . | title | quote }}
    {{- end }}
```


Flow control — **range** with **\$**

Use **\$** to access parent values inside **range** :

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ .Release.Name }}-configmap
data:
  myvalue: "Hello World"
  {{- with .Values.favorite }}
  drink: {{ .drink | default "tea" | quote }}
  food: {{ .food | upper | quote }}
  toppings: |-
    {{- range $.Values.pizzaToppings }}
    - {{ . | title | quote }}
    {{- end }}
  {{- end }}
```

\$ is mapped to the root scope when template execution begins and does not change

Template lists — tuple

Create an inline list with `tuple` and iterate:

```
sizes: |-  
  {{- range tuple "small" "medium" "large" }}  
  - {{ . }}  
  {{- end }}
```

Output:

```
sizes: |-  
- small  
- medium  
- large
```

Variables

Capture a value before entering a scoped block:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ .Release.Name }}-configmap
data:
  myvalue: "Hello World"
  {{- $relname := .Release.Name -}}
  {{- with .Values.favorite }}
  drink: {{ .drink | default "tea" | quote }}
  food: {{ .food | upper | quote }}
  release: {{ $relname }}
  {{- end }}
```

Variables use `:=` assignment operator. `$` always refers to the root scope.

Variables — \$ in range

```
{{- range .Values.tlsSecrets }}
apiVersion: v1
kind: Secret
metadata:
  name: {{ .name }}
  labels:
    app.kubernetes.io/name: {{ template "fullname" $ }}
    helm.sh/chart: "{{ $Chart.Name }}-{{ $Chart.Version }}"
    app.kubernetes.io/instance: "{{ $Release.Name }}"
    app.kubernetes.io/version: "{{ $Chart.AppVersion }}"
    app.kubernetes.io/managed-by: "{{ $Release.Service }}"
type: kubernetes.io/tls
data:
  tls.crt: {{ .certificate }}
  tls.key: {{ .key }}
---
{{- end }}
```

\$ always points to root context — use it inside `range` and `with` to access top-level values

Part 3: Best Practices

Partials, naming, docs, values & packaging hygiene

Named templates — **define**

```
{{- define "MY.NAME" }}  
  # body of template here  
{{- end }}
```

Define a reusable labels block:

```
{{- define "mychart.labels" }}  
  labels:  
    generator: helm  
    date: {{ now | htmlDate }}  
{{- end }}
```

Named templates — embedding

```
{{- define "mychart.labels" }}
  labels:
    generator: helm
    date: {{ now | htmlDate }}
{{- end }}

apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ .Release.Name }}-configmap
  {{- template "mychart.labels" }}
data:
  myvalue: "Hello World"
  {{- range $key, $val := .Values.favorite }}
  {{ $key }}: {{ $val | quote }}
  {{- end }}
```

Prefer `include` over `template` for better YAML formatting control

Partials — define and use

Defining a partial:

```
{{- define "mychart.labels" -}}
labels:
  app: {{ .Values.app.name }}
  version: {{ .Values.app.version }}
{{- end -}}
```

Using a partial:

```
metadata:
  {{ include "mychart.labels" . | indent 4 }}
```

Partials are reusable templates that help organize and deduplicate code across templates.

files — helper templates

Files starting with  are not rendered as manifests — they store partials and helpers:

```
{{- define "mychart.fullname" -}}
{{- printf "%s-%s" .Release.Name .Chart.Name | trunc 63 | trimSuffix "-" -}}
{{- end -}}
```

Using from any template:

```
metadata:
  name: {{ include "mychart.fullname" . }}
```

Files with  prefix are available everywhere within chart templates but produce no Kubernetes objects

Best practices for partials

- **Organize partials** — use `_helpers.tpl` or other `_` prefixed files to keep templates clean and maintainable
- **Naming conventions** — use a consistent naming convention to avoid conflicts and improve readability
- **Indentation** — use `indent` and `nindent` functions to ensure proper YAML formatting when including partials

By using partials and `_` files, you can create modular, reusable, and maintainable Helm charts

include vs template

include (preferred)

Returns result as string — allows `indent` / `nindent` for YAML formatting

```
{{ include "mychart.labels" . | indent 4 }}
```

Advantages: full output formatting control; works correctly in pipelines

Prefer `include` — the output formatting can be handled better for YAML documents

template

Executes in-place — no formatting control

```
{{ template "mychart.labels" . }}
```

Disadvantages: cannot manipulate output formatting; can break YAML indentation

Naming conventions — template helpers

Practice	Example
Prefix with chart name	<code>mychart.labels</code> , <code>mychart.fullname</code>
Versioning when needed	<code>mychart.v1.labels</code> , <code>mychart.v2.labels</code>
Consistent camelCase	<code>mychart.serviceAccountName</code>
No special characters	Avoid <code>!</code> , <code>@</code> , spaces in names

```
{{ define "mychart.serviceAccountName" }}
{{ printf "%s-%s" .Release.Name .Chart.Name | trunc 63 | trimSuffix "-" }}
{{ end }}
```

Template functions — `default` and `required`

```
# default – fallback when value is empty
data:
  environment: {{ .Values.environment | default "dev" | quote }}
  region: {{ .Values.region | upper | quote }}
```

```
# required – fail fast with a clear error
required "A valid foo is required!" .Bar

metadata:
  name: {{ required "Name is required" .Values.configName }}
```

Document your charts — README.md

Include a `README.md` with overview and usage:

```
# MyChart

## Overview
MyChart is a Helm chart for deploying a sample application.

## Installation
To install the chart with the release name `my-release`:

helm install my-release ./mychart
```

Document your charts — NOTES.txt

NOTES.txt provides post-install instructions (supports templating):

```
{{- if .Values.service.type == "LoadBalancer" }}  
1. Get the application URL by running these commands:  
  export SERVICE_IP=$(kubectl get svc --namespace {{ .Release.Namespace }} \  
    {{ include "mychart.fullname" . }} \  
    -o jsonpath='{.status.loadBalancer.ingress[0].ip}')  
  echo http://$SERVICE_IP:{{ .Values.service.port }}  
{{- else if .Values.service.type == "ClusterIP" }}  
1. Get the application URL by running these commands:  
  kubectl port-forward $POD_NAME 8080:80  
  echo "Visit http://127.0.0.1:8080 to use your application"  
{{- end }}
```

Content of NOTES.txt is displayed after every helm install and helm upgrade

Naming conventions — charts & values

Valid chart names (lowercase, hyphens only)

- `nginx`
- `wordpress`
- `wordpress-on-nginx`

Invalid

- `Nginx` (capital letter)
- `wordpress0nNginx` (camelCase)
- `wordpress_on_nginx` (underscore)

values.yaml — use camelCase

```
replicaCount: 3
wordpressUsername: user
wordpressSkipInstall: false
```

Pass values at install:

```
# file
helm install -f myvals.yaml ./mychart
# individual param
helm install --set foo=bar ./mychart
```


`.helmignore`

```
# Match any file or path named .git
.git

# Match any text file
*.txt

# Match only directories named mydir
mydir/

# Match only text files in the top-level directory
/*.txt

# Match only the file foo.txt in the top-level directory
/foo.txt

# Match any file named ab.txt, ac.txt, or ad.txt
a[b-d].txt

# Match any file under subdir matching temp*
*/temp*

**/temp*
temp?
```

Flat vs nested values

Nested (logical grouping)

```
image:  
  repository: nginx  
  pullPolicy: IfNotPresent  
  tag: "1.16.0"
```

Flat (simpler overrides)

```
imageRepository: nginx  
imagePullPolicy: IfNotPresent  
imageTag: "1.16.0"
```

Nested values are preferred for readability; flat values are easier to override with `--set`

Quote strings

```
enabled: false    # boolean false
enabled: "false"  # string "false"
```

Always wrap your string values between `"` quote signs.

YAML auto-casts unquoted values to boolean, integer, or null — `"false"` and `false` assign different types

Part 4: Advanced Patterns

Rollout tricks, hooks, versioning, OCI, safe CD

Update deployments on config change

Any change in a ConfigMap triggers a rolling restart:

```
kind: Deployment
spec:
  template:
    metadata:
      annotations:
        checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}
```

Kubernetes only restarts Pods when the Pod spec changes — this annotation makes ConfigMap changes visible to the Deployment controller

Opt out of resource deletion

Keep resources in the cluster even after `helm uninstall`:

```
kind: Secret
metadata:
  annotations:
    "helm.sh/resource-policy": keep
```

Without this annotation, all chart resources are deleted on `helm uninstall`

Randomly generate values

Generate once, persist on upgrade using `lookup`:

```
{{- $rootPasswordValue := (randAlpha 16) | b64enc | quote }}
{{- $secret := (lookup "v1" "Secret" .Release.Namespace "db-keys") }}
{{- if $secret }}
{{- $rootPasswordValue = index $secret.data "root-password" }}
{{- end -}}
apiVersion: v1
kind: Secret
metadata:
  name: db-keys
  namespace: {{ .Release.Namespace }}
type: Opaque
data:
  root-password: {{ $rootPasswordValue }}
```

`lookup` reads the existing secret on upgrade so passwords are not regenerated each time

Chart vs application versioning

Chart version

What changes when the packaging, templates, or defaults change

``Chart.yaml: version``

SemVer for the package itself

Bump on chart structure or behavior changes

App version

What image or application code is actually deployed

``appVersion`` in ``Chart.yaml``

Usually matches the container image tag

Bump when the app release changes

OCI registries and ArtifactHub

Recommended flow

Write chart in Git

Package it with ``helm package``

Push to an OCI registry with ``helm push``

Discover it in ArtifactHub or pull it directly

Key rules

Prefer OCI for chart distribution

Pin dependency versions explicitly

Helm 4 — pin by digest (`@sha256:...`) for supply-chain safety

Use ArtifactHub for discovery, not as the source of truth

Helm Chart Hooks — use cases

``pre-install``

Prepare secrets or config before main resources are created

``post-install``

Run migrations or bootstrap checks after the release exists

``post-upgrade``

Kick off smoke tests after the upgrade finishes

``pre-delete``

Clean up external systems before uninstalling the release

Hook example

```
apiVersion: batch/v1
kind: Job
metadata:
  annotations:
    # This is what defines this resource as a hook. Without this line,
    # the job is considered part of the release.
    "helm.sh/hook": post-install, post-upgrade
    "helm.sh/hook-weight": "-5"
    "helm.sh/hook-delete-policy": before-hook-creation, hook-succeeded
```

Test hooks use `helm.sh/hook: test` and run with `helm test`

Idempotent CD pipelines

Two essential rules:

- Always use `helm upgrade --install` — installs if not present, upgrades if it is
- Use `--atomic` and `--wait` — rolls back automatically on failed upgrade and waits for readiness (*Helm 4: `--atomic` is deprecated* → `--rollback-on-failure`)
- Prefer `--create-namespace` for fresh environments
- Helm 4: `--wait=legacy` falls back to v3 readiness logic for charts with broken `livenessProbe`

```
helm upgrade --install my-release ./mychart \  
  --atomic \  
  --wait \  
  --timeout 5m \  
  --create-namespace \  
  --namespace production
```

Part 5: Helm 4 — what's new

Major version released as v4.0; current stable is v4.2.x

Why a new major version?

Helm 3 shipped in 2019 — a lot of pain accumulated:

- Plugins were arbitrary shell scripts → no sandbox, no signing, supply-chain risk
- Client-side apply lost fields touched by **HPA**, operators, mutating webhooks
- OCI distribution was tagged-only → tag re-pushes broke "immutable" pipelines
- `values.yaml` grew to thousands of lines with no way to split it
- Resource readiness waited on `kind: Deployment` only — CRs went untracked

Helm 4 takes a swing at all five — and keeps **chart v2 unchanged** so existing charts still install

Plugin system overhaul (HIP-0026)

Problem: old plugins were shell scripts — anyone could `helm plugin install` and run code with your kube-context.

Helm 4 solution:

- New `extism/v1` WebAssembly runtime → sandboxed execution
- Plugins are packaged, **signed, and verified** before install
- Three plugin types: **CLI, getter, post-renderer**
- Installers: tarball, HTTP, local directory, **OCI registry**
- Custom **template functions** can now ship via plugins

```
# Install a signed plugin from OCI
helm plugin install oci://ghcr.io/myorg/plugins/vault-getter:1.0.0
helm plugin verify vault-getter
```

Server-side apply by default (HIP-0023)

Problem: client-side apply diffs against the **last-applied** annotation. If HPA changed `replicas` or an operator mutated a field, Helm overwrote it on next upgrade.

Helm 4 solution:

- New releases default to **Server-Side Apply (SSA)**
- Kubernetes tracks **field ownership** per manager
- Helm only changes fields it owns; HPA/operators keep their fields
- Adoption of pre-existing resources is now safe (no "already exists" errors)

```
# Mark resources as adoptable for SSA conflicts
metadata:
  annotations:
    helm.sh/resource-policy: keep
    meta.helm.sh/release-name: my-release
```


OCI digest pinning

Problem: `oci://registry/mychart:1.2.3` is a **tag** — tags can be re-pushed. Yesterday's "1.2.3" $\neq$ today's "1.2.3".

Helm 4 solution: pin to the **content digest**:

```
# Pull / install by digest – immutable reference
helm install myapp oci://registry.example.com/charts/myapp@sha256:abc123...

# Inspect digest of a published chart
helm show chart oci://registry.example.com/charts/myapp:1.2.3 --digest
```

- Same model as Docker / containerd image digests
- Combine with **cosign** signatures for full supply-chain attestation
- `Chart.yaml` dependencies can also reference digests

Multi-document values files

Problem: large charts end up with monolithic 1000-line `values.yaml` — diffs are painful, ownership is unclear.

Helm 4 solution: YAML supports multiple documents in one file; Helm now **merges them in order**:

```
# values.yaml
# --- Document 1: image
image:
  repository: nginx
  tag: "1.27"
---
# --- Document 2: resources (managed by SRE)
resources:
  requests:
    cpu: 100m
    memory: 128Mi
---
# --- Document 3: feature flags (managed by app team)
features:
  newCheckout: true
```

Each `---` block is rendered separately, then merged top-to-bottom — last document wins on conflicts

must* functions and JSON Schema 2020

Problem: silent failures in templates were the #1 source of "chart installed but app broken" incidents.

Helm 4 solution — fail loudly:

```
# Old: returns empty string on failure, deploys broken YAML
data: {{ toYaml .Values.config }}

# New: fails the install with a clear error
data: {{ mustToYaml .Values.config }}
```

New functions: `mustToYaml`, `mustToJson`, `mustToToml`, `mustFromJson`, ...

Plus **JSON Schema 2020** support in `values.schema.json`:

- `$dynamicRef`, conditional schemas, content validation
- Full IDE auto-completion and validation against modern tooling

kstatus — universal readiness tracking

Problem: `--wait` only knew about `Deployment`, `StatefulSet`, `DaemonSet`. Custom Resources from operators (Argo, Istio, Crossplane) were ignored — release reported "ready" before the app was actually up.

Helm 4 solution: integrates **kstatus** — Kubernetes' generic status convention:

- Tracks **any** Kind that follows the `status.conditions` convention
- Honours `kstatus` annotations on CRDs
- Granular: per-kind tracking via `trackKinds` / `skipKinds`

```
# Wait for the Argo Application AND the underlying Deployment
helm upgrade --install myapp ./chart --wait --timeout 5m
```

Breaking changes — what to fix in your charts

Change	Old (Helm 3)	New (Helm 4)
Post-renderer	external binary path	named plugin
Registry login	full URL with scheme	domain only
Deprecated flags	<code>--create-pods</code> , <code>--client</code>	removed
Plugin packaging	shell scripts	<code>apiVersion: v1</code> manifest
Charts	v1 (Helm 2) supported	v1 removed ; v2 supported, v3 experimental

Use the official `helm migrate` plugin to scan a repo for breaking patterns

Compatibility & adoption strategy

Stays working as-is

- All chart v2 charts (your existing ones)
- Existing `helm install` / `upgrade` commands
- Existing `values.yaml` files
- ArtifactHub, OCI registries, Helm repos

Opt-in / new

- Server-side apply for new releases
- WASM plugins (old plugins must repackage)
- Chart v3 (experimental, `HELM_EXPERIMENTAL_CHART_V3=1`)
- Multi-document values
- Digest pinning, `must*` functions

Helm 3 → Helm 4 is **not** a "stop-the-world" upgrade — migrate at your own pace, feature by feature

Time to make hands dirty — Lab 05

Create your first Helm chart

Соберём свой первый chart и посмотрим, как `values.yaml` и `templates` превращаются в манифесты.

[helm_chart_lab.md](#) · [all Helm labs](#) →

Итоги по Helm

Что умеете

- Теперь вы умеете собирать приложение в `chart`
- Helm снижает ручную работу и уменьшает число ошибок при деплое
- `values.yaml`, шаблоны и релизы дают повторяемость и контроль

Следующий шаг

- Сделайте свой первый `chart`
- Прогоните его в кластере
- Смотрите, как Helm упрощает обновление и откат

